

Family Context Agent

Product, Architecture & Implementation Blueprint

Hackathon: Nebius × NVIDIA Global AI Hackathon

Target: Working, deployed, judge-testable product

Internal completion target: October 25, 2026

Audience: Complete beginners through CS students

Design principle: Simple explanation, professional architecture.

This blueprint converts the existing Family Context Agent concept into an implementation plan. The product goal remains: accept natural-language messages and household documents, turn them into structured expenses and commitments, maintain household state, understand deadlines and dependencies, ask when information is missing, prioritize attention, follow up, and close completed commitments.

1. What Are We Building?

A family has information scattered everywhere:

Bills

Expenses

Appointments

College deadlines

Repairs

Documents

Renewals

Family plans

The **Family Context Agent** creates one intelligent layer over this information.

The simple product vision is:

The family tells the system what is happening. The system remembers what matters, connects related information, and helps the family know what needs attention next.

Example

User:

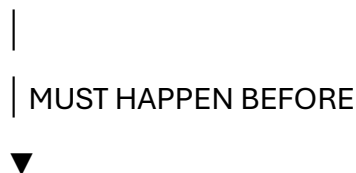
Grandma's doctor review is October 18. Doctor asked her to complete CBC before the appointment.

The application should understand:

Person: Grandma

CBC Test

Status: Pending



Doctor Review

Date: October 18

If October 16 arrives and CBC is still pending:

High Priority: Grandma's CBC is still pending and should be completed before her doctor review on October 18.

This dependency reasoning is one of the capabilities that makes the product more than a reminder application.

2. MVP Scope

We will build **four main areas**.

Area	Examples
Expenses	Grocery, fuel, transport, repairs
Bills & Renewals	Electricity, internet, insurance
Family Commitments	Appointment, lab test, plumber
Education	Project, assignment, fee, examination

This follows the focused four-category MVP already defined in the concept.

Inputs

For the hackathon:

Input	MVP
Natural-language text	✓
PDF	✓
Image	✓
Voice	✗
WhatsApp	✗
Email integration	✗
Bank/UPI	✗
IoT	✗

For PDF/image support, we should initially handle only a few known document types rather than claiming to understand every household document. The original concept recommends exactly this controlled approach.

3. User Experience

We should build a **responsive web application** that works on laptop and mobile browsers.

The application should have five main areas:

HOME

CHAT

TIMELINE

EXPENSES

DOCUMENTS

Do not build a complex mobile app for the hackathon.

3.1 Home Dashboard

This should be the first screen.

Family Context	Nair Family
----------------	-------------

Good Morning 🙌		
3 things need your attention		
HIGH		
Grandma's CBC is still pending		
Required before doctor review · Oct 18		
MEDIUM		
Electricity Bill · ₹3,240		
Due in 3 days · Unpaid		
Upcoming	Expenses	Recent Activity
[Ask Family Agent]		

The most important part is **Things needing attention**.

The existing concept makes this "What should we handle next?" experience central to the product.

3.2 Chat

Family Agent	

You

₹1,850 groceries from Lulu today.

Agent

Added ₹1,850 to groceries for today.

You

What should we handle next?

Agent

Grandma's CBC needs attention first...

Tell me what's happening...  Send

The attachment button accepts PDF/image.

3.3 Timeline

Show commitments chronologically.

OCTOBER

15 CBC Test

Grandma

Pending

18 Doctor Review

Grandma

Upcoming

20 College Fee

Rahul

₹12,000

25 AI Project

Rahul

In Progress

3.4 Expenses

Show simple useful information:

THIS MONTH

Total recorded: ₹28,450

Groceries ₹8,450

Transport ₹3,200

Utilities ₹4,800

Education ₹12,000

These totals must come from database calculations, **not LLM arithmetic.**

3.5 Documents

Show uploaded documents and their processing state:

Electricity_Oct.pdf

Type: Electricity Bill

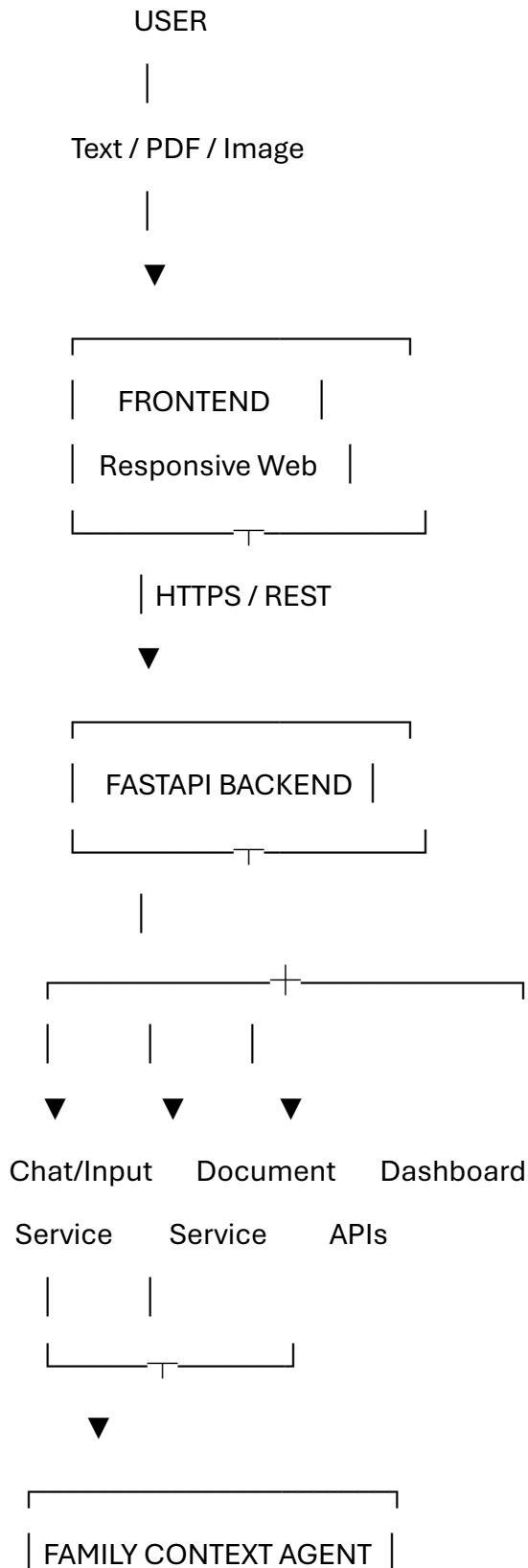
Amount: ₹3,240

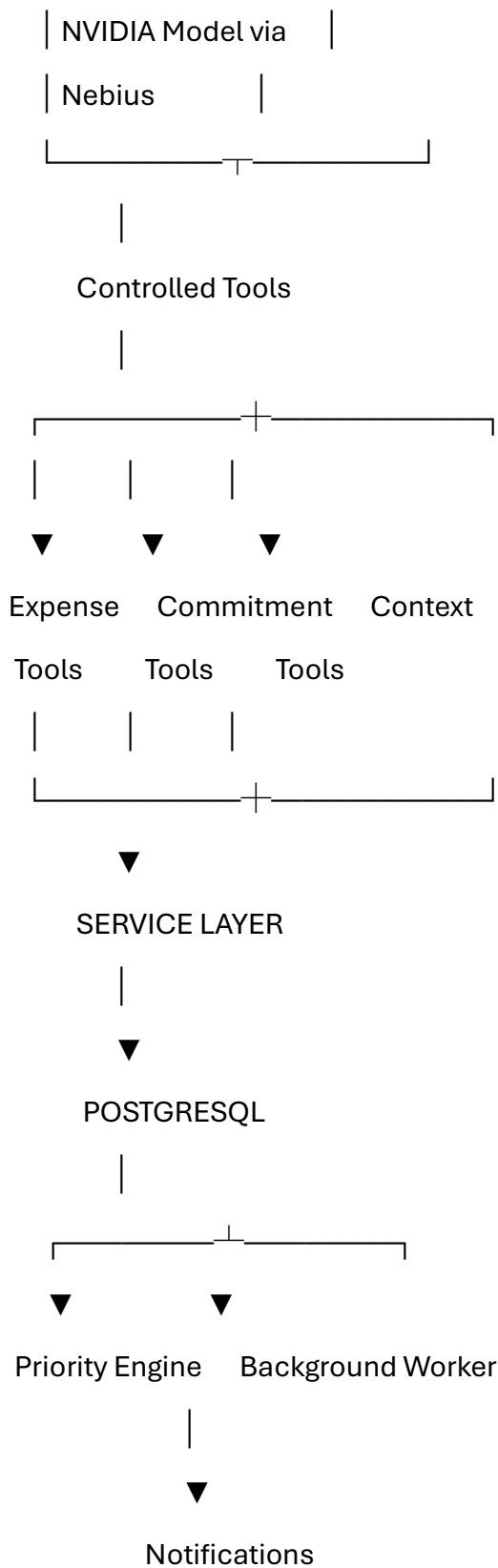
Due: October 18

Status: Processed

4. Complete High-Level Architecture

This is the architecture we should initially target.





Documents —————> OBJECT/FILE STORAGE

Important principle

The LLM should not directly modify PostgreSQL.

The LLM requests controlled actions through tools. The application validates and performs them.

That gives us:

LLM = intelligence

Tools = controlled actions

PostgreSQL = reliable state

Backend = control layer

5. Recommended Technology Stack

We should **freeze the stack early** instead of asking students to research ten alternatives.

Component	Recommendation
Frontend	React
UI styling	Tailwind CSS
Backend	Python + FastAPI
Validation	Pydantic
ORM	SQLAlchemy
Database	PostgreSQL
Migration	Alembic
Agent orchestration	Lightweight custom orchestration first; LangGraph only where it provides clear state/workflow value
AI	NVIDIA open model through Nebius
Files	Object/file storage

Component	Recommendation
Background jobs	Lightweight scheduler initially
API style	REST
Testing	Pytest
API testing	Swagger/OpenAPI + Postman if useful
Version control	Git + GitHub
Containers	Docker
Deployment	Cloud-hosted frontend/backend/database; final provider selected after validating hackathon/Nebius options

Why FastAPI?

Because our AI/backend work is Python-based and FastAPI gives us APIs, validation, async support and automatic API documentation without unnecessary complexity.

Why PostgreSQL?

Our information has strong relationships:

Family

↓

Members

↓

Commitments

↓

Dependencies

and:

Family

↓

Expenses

PostgreSQL handles this structured relational information very well.

Why React?

We want a polished interactive product UI rather than an interface that looks primarily like a development prototype.

6. How One Input Travels Through the Application

This is one of the most important sections for beginners.

User enters:

₹1,850 groceries from Lulu today.

Step 1 — Frontend

React receives the message.

Step 2 — API

Frontend sends something similar to:

POST /api/v1/chat

with:

```
{  
  "family_id": "demo-family",  
  "member_id": "rahul",  
  "message": "₹1,850 groceries from Lulu today."  
}
```

Step 3 — Backend

FastAPI validates the request.

Step 4 — Agent

Backend gives the message and relevant context to the Family Context Agent.

Step 5 — NVIDIA model

The model understands:

```
{  
  "intent": "record_expense",  
  "amount": 1850,  
  "category": "groceries",
```

```
"merchant": "Lulu",  
"date": "today"  
}
```

Step 6 — Tool selection

Agent requests:

```
add_expense(...)
```

Step 7 — Validation

Backend validates amount, family/member IDs, date and other fields.

Step 8 — Database

Expense is written to PostgreSQL.

Step 9 — Response

Backend returns success.

Step 10 — UI

Chat shows:

Added ₹1,850 under Groceries for today.

Expenses dashboard now reflects the new value.

7. Database Design

PostgreSQL becomes the application's **source of truth**.

Do not depend on LLM chat history as the source of truth.

Core tables

families

family_members

expenses

commitments

dependencies

documents

notifications

conversation_messages

agent_events

7.1 Families

id

name

created_at

Example:

family_001 | Nair Family

7.2 Family Members

id

family_id

name

relationship

role

created_at

7.3 Expenses

id

family_id

member_id

amount

category

merchant

expense_date

description

source

created_at

7.4 Commitments

This is one of our most important tables.

id

family_id

member_id

type

title

description

status

priority

start_date

due_date

completed_at

source

created_at

updated_at

Possible status values:

PENDING

SCHEDULED

IN_PROGRESS

COMPLETED

OVERDUE

CANCELLED

7.5 Dependencies

This creates connected context.

id

family_id

source_commitment_id

target_commitment_id

relationship_type

created_at

Example:

CBC

MUST_COMPLETE_BEFORE

Doctor Review

This table is extremely important for differentiation.

7.6 Documents

id

family_id

member_id

file_name

storage_path

document_type

processing_status

extracted_data

created_at

7.7 Notifications

id

family_id

commitment_id

priority

message

reason

status

created_at

resolved_at

7.8 Agent Events

Useful for observability:

USER_MESSAGE_RECEIVED
INTENT_DETECTED
TOOL_CALLED
COMMITMENT_CREATED
DEPENDENCY_CREATED
DOCUMENT_PROCESSED
CLARIFICATION_REQUESTED
NOTIFICATION_CREATED
ERROR

8. Where Should PDFs and Images Be Stored?

Do not normally put entire PDF/image files directly inside ordinary PostgreSQL fields.

Use:

File

↓

Object/File Storage

and PostgreSQL stores:

file name

storage location

document type

processing status

extracted information

Conceptually:

PostgreSQL

|

└ "Electricity_Oct.pdf is stored at X"

Storage

|
└ actual Electricity_Oct.pdf

For local development, this can initially be a local uploads directory. Deployment should move this to suitable persistent/object storage.

9. Family Context Agent

We should have **one primary agent**, not ten agents.

The concept explicitly recommends one Family Context Agent with structured state, meaningful tools and deterministic scheduling rather than creating separate agents for every family function.

The agent's responsibilities:

Understand user input

↓

Identify intent

↓

Extract important information

↓

Retrieve relevant household context

↓

Detect ambiguity

↓

Determine action

↓

Choose tool

↓

Receive tool result

↓

Reason about relationships

↓

Explain result

The agent should NOT:

directly edit database

invent missing information

diagnose diseases

give investment advice

silently perform uncertain actions

calculate authoritative financial totals from memory

10. Agent Tools

Tools are controlled functions available to the agent.

We should keep approximately **6–8 meaningful tools**, rather than dozens.

Expense tools

add_expense()

get_expenses()

get_expense_summary()

Commitment tools

create_commitment()

update_commitment()

search_commitments()

Context tool

get_family_context()

Priority tool

get_current_priorities()

Document processing can be exposed through the document service/tool boundary as appropriate.

The important principle is:

User

↓

Agent

↓

"I need to record an expense."

↓

Expense Tool

↓

Application Service

↓

Database

not:

Agent

↓

Random SQL generated by LLM

↓

Database

11. Structured Output

Whenever possible, ask the model for a defined structure.

For example:

```
{  
  "intent": "create_commitment",  
  "commitment_type": "bill",  
  "title": "Electricity Bill",  
  "amount": 3240,  
  "due_date": "2026-10-18",  
  "status": "pending",  
  "missing_fields": [],
```

"confidence": 0.94

}

Pydantic validates this before our software acts.

If required information is missing:

{

"intent": "create_commitment",

"amount": 2000,

"purpose": null,

"requires_clarification": true

}

Agent:

What was the ₹2,000 payment for?

The existing testing plan explicitly says the system should ask rather than guess when given an ambiguous payment.

12. Memory and State

There are different meanings of "memory."

Conversation memory

Useful for the current conversation.

Example:

"What about that bill?"

Household state

Important long-term facts stored in PostgreSQL.

Example:

Electricity Bill

₹3,240

Due Oct 18

Status PENDING

Persistent commitments

Information that may matter for weeks/months.

Example:

Vehicle insurance expires December 15.

Temporary information

Example:

Need milk tomorrow.

The concept already identifies this difference between temporary information and persistent commitments.

Important rule

LLM memory is not our database.

If the backend restarts, important household information must remain available.

13. Reference Resolution

Suppose:

Electricity Bill → ₹3,240

Internet Bill → ₹1,499

User says:

I paid the electricity bill.

Easy.

But if user says:

I paid it.

The agent must inspect relevant recent context.

If only one clear candidate exists, it may resolve it.

If there are multiple reasonable candidates:

Do you mean the electricity bill or internet bill?

The concept explicitly requires safe ambiguity handling of this kind.

14. Dependency Engine

This is one of our flagship features.

Example:

CBC

↓ MUST_COMPLETE_BEFORE

Doctor Review

Another:

Implementation

↓

Testing

↓

Documentation

↓

Submission

Store these relationships explicitly.

The system can then reason:

Submission approaching

+

Testing incomplete

+

Testing depends on implementation

↓

Project schedule is at risk

This is much stronger than generating independent reminders.

15. Priority Engine

We should **not ask the LLM alone to invent priorities**.

Use a hybrid approach:

Deterministic evidence

+

Dependency information

+

Contextual AI reasoning where useful

↓

Priority

↓

Human-readable explanation

Relevant signals could include:

Time remaining

Overdue status

Incomplete prerequisite

Dependency impact

User-defined importance

Existing completion status

For example:

CBC

Due very soon

Still pending

Required before doctor review

→ HIGH

The important product behaviour is not merely assigning a label. The application should show the **reason**.

16. Background Processing — Does the Agent Need Cron?

Yes, some form of scheduled/background processing is useful.

But **the LLM does not need to run continuously.**

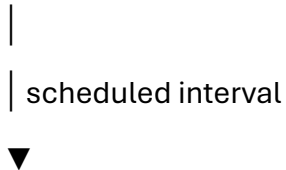
Imagine nobody opens the application today.

Tomorrow the electricity bill becomes due in two days.

Something should update the attention state.

A scheduled worker can run periodically:

SCHEDULER



Find active commitments



Check deadlines



Check dependencies



Check overdue state



Recalculate priorities



Create/update notifications

Most checks are deterministic.

Only invoke the LLM when genuine contextual reasoning or natural-language generation adds value.

Important

For the hackathon we don't need enterprise-scale Kafka/Celery infrastructure merely to look sophisticated.

Start with a reliable lightweight scheduler/background-worker design. Keep the code structured so it can later move to a stronger job system.

17. PDF/Image Processing

Example: user uploads electricity bill.

PDF/Image

↓

Frontend

↓

Upload API

↓

File validation

↓

File Storage

↓

Document Processor

↓

Text/visual understanding

↓

NVIDIA model

↓

Structured extraction

Possible result:

```
{  
  "document_type": "electricity_bill",  
  "provider": "Demo Electricity Board",  
  "amount": 3240,  
  "due_date": "2026-10-18"  
}
```

Then **do not immediately trust every extraction.**

Show:

I found an electricity bill for ₹3,240 due October 18. Would you like me to add it?

User:

Yes.

Then:

Commitment created

↓

Database updated

↓

Priority engine

↓

Dashboard updated

This human-confirmation flow is already part of the concept.

18. NVIDIA + Nebius Integration

The NVIDIA model must provide **real intelligence**, not exist as a logo on the architecture diagram.

Its useful responsibilities include:

Natural-language understanding

Structured information extraction

Intent understanding

Document understanding where supported

Reference resolution

Missing-information detection

Dependency understanding

Tool selection

Contextual reasoning

Natural-language explanation

The concept already assigns these kinds of responsibilities to the NVIDIA/Nebius layer.

The integration pattern should be:

FastAPI

↓

AI Provider/Model Client

↓

Nebius

↓

NVIDIA model

↓

Structured response/tool request

↓

FastAPI application

Important engineering decision

Do not spread direct model API calls throughout the application.

Create one abstraction such as:

AIModelClient

Then services/agent code uses that client.

That makes configuration, testing, logging and model changes easier.

19. Backend API

Recommended API structure:

/api/v1/chat

/api/v1/families

/api/v1/members

/api/v1/expenses

/api/v1/expenses/summary

/api/v1/commitments

/api/v1/commitments/upcoming

/api/v1/documents

/api/v1/documents/upload

/api/v1/priorities

/api/v1/timeline

/api/v1/demo/scenarios

Do not create an API for every tiny operation.

The backend owns business logic.

The frontend should not contain important business rules.

20. Repository Structure

Use one clean repository:

family-context-agent/

|

| └─ frontend/

| | └─ src/

| | | └─ components/

| | | └─ pages/

| | | └─ services/

| | | └─ hooks/

| | └─ ...

|

```
└─ backend/
|  └─ app/
|  |  └─ api/
|  |  └─ agents/
|  |  └─ tools/
|  |  └─ services/
|  |  └─ repositories/
|  |  └─ models/
|  |  └─ schemas/
|  |  └─ jobs/
|  |  └─ core/
|  |  └─ main.py
|  |
|  └─ tests/
|
└─ synthetic_data/
|
└─ demo_documents/
|
└─ evaluation/
|
└─ deployment/
|
└─ docs/
|
└─ docker-compose.yml
└─ .env.example
```

└─ README.md

└─ LICENSE

Why separate layers?

API

↓

Service

↓

Repository

↓

Database

API = communicates with frontend.

Service = business logic.

Repository = database access.

This prevents all application logic from ending up in one app.py.

21. Security and Privacy

This product deals with potentially sensitive family information.

For the hackathon, use a **fictional household**.

Never put API keys into GitHub.

Use:

.env

and commit:

.env.example

not the real .env.

Environment variables might include:

DATABASE_URL=

NEBIUS_API_KEY=

MODEL_NAME=

STORAGE_CONFIG=

Validate uploaded files.

Restrict supported formats and file sizes.

Do not execute uploaded files.

For a future product, family-member permissions become important because not every family member should necessarily see every financial/private document. The concept already identifies this requirement even though a complex permission model is outside the MVP.

22. Guardrails

Medical

Allowed:

CBC remains pending before Grandma's doctor appointment.

Not allowed:

Your CBC result means you have disease X.

Financial

Allowed:

₹17,000 of recorded commitments are due this week.

Not:

You should take a loan to pay them.

These boundaries are already explicitly defined in the concept.

23. Failure Handling

A top-quality demo should also behave well when things fail.

Document cannot be understood

I couldn't confidently identify the due date. Please enter it manually.

Expense category unknown

I couldn't determine the category. Would you like to select one?

Ambiguous reference

Do you mean the electricity bill or internet bill?

AI unavailable

The application should still show already stored commitments and expenses.

Database unavailable

Return a clean error.

Never pretend the operation succeeded.

These failure behaviours are already part of the project's intended production-quality thinking.

24. Observability

We need logs.

Example:

request_id=123

event=USER_MESSAGE_RECEIVED

event=INTENT_DETECTED

intent=create_expense

event=TOOL_CALLED

tool=add_expense

event=EXPENSE_CREATED

latency=1.42s

Useful events include user input received, tool selected, structured data created, commitments/dependencies created, human confirmations, errors and latency.

Do **not** display hidden chain-of-thought.

Show useful system activity instead:

✓ Message understood

- ✓ Expense recorded
 - ✓ Household state updated
-

25. Synthetic Data

Do not use random unrelated rows.

Create a **connected fictional household history**.

For example:

Nair Family

Lakshmi

└— CBC

└— Doctor Review

Rahul

└— AI Project

└— Testing Milestone

└— College Fee

Household

└— Electricity

└— Internet

└— Groceries

└— Kitchen Repair

Create perhaps 2–3 months of coherent data.

The concept already recommends connected synthetic scenarios rather than random data.

26. Testing Strategy

We need three broad layers.

Unit Tests

Test individual logic:

date calculations

priority rules

validation

expense calculations

status transitions

Integration Tests

Test components together:

API → Service → PostgreSQL

Agent → Tool → Database

Document → Extraction → Confirmation → Commitment

Agent Scenario Tests

These are extremely important.

Example:

INPUT

"Electricity bill ₹3,240 due October 18."

EXPECTED

Type = Bill

Amount = 3240

Due = Oct 18

Status = Pending

Commitment created = Yes

The concept already proposes ground-truth evaluation rather than subjective "the answer looks good" testing.

27. Evaluation Metrics

Track useful measures such as:

Information extraction accuracy

Commitment detection accuracy

Dependency detection accuracy

Completion resolution accuracy

Duplicate prevention

Priority correctness

False reminder rate

Clarification behaviour

Hallucination rate

Tool-selection accuracy

These align with the evaluation areas already identified for the project.

Do not claim:

99% accurate

unless we actually ran an evaluation that produced that number.

28. Judge Mode

This is a major feature.

Judges should not need to spend two weeks creating household history.

Add something like:

DEMO FAMILY

Nair Family

Choose Scenario

- ☐ Busy Family Week
- ☐ Upcoming Medical Review
- ☐ Multiple Bills
- ☐ Student Deadline
- ☐ Budget Pressure
- ☐ Missed Commitment
- ☐ Conflicting Information

[Load Scenario]

This approach is already proposed in the concept specifically to make judge testing easy.

29. Judge Test Journey

A strong demonstration:

Step 1

Judge loads:

Busy Family Week

Existing state:

Electricity bill → Oct 18

CBC → Pending

↓

Doctor Review → Oct 20

AI Project → Oct 25

Kitchen repair → Oct 19

Step 2

Judge asks:

What should we handle first?

Agent explains priorities.

Step 3

Judge says:

CBC completed today.

Agent updates the correct commitment.

Step 4

Judge asks:

What needs attention now?

CBC should disappear from pending priorities.

Step 5

Judge uploads an electricity bill.

System extracts:

Amount

Due date

Type

and asks for confirmation.

Step 6

Judge confirms.

Database changes.

Dashboard changes.

Within a few minutes, the judge has observed:

NVIDIA model

Natural language understanding

Tool calling

Persistent state

Memory

Reference resolution

Dependency reasoning

Priority reasoning

Document understanding

Human-in-the-loop

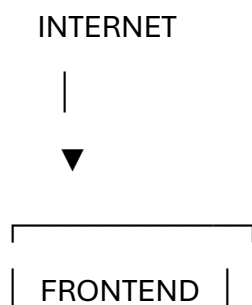
Database integration

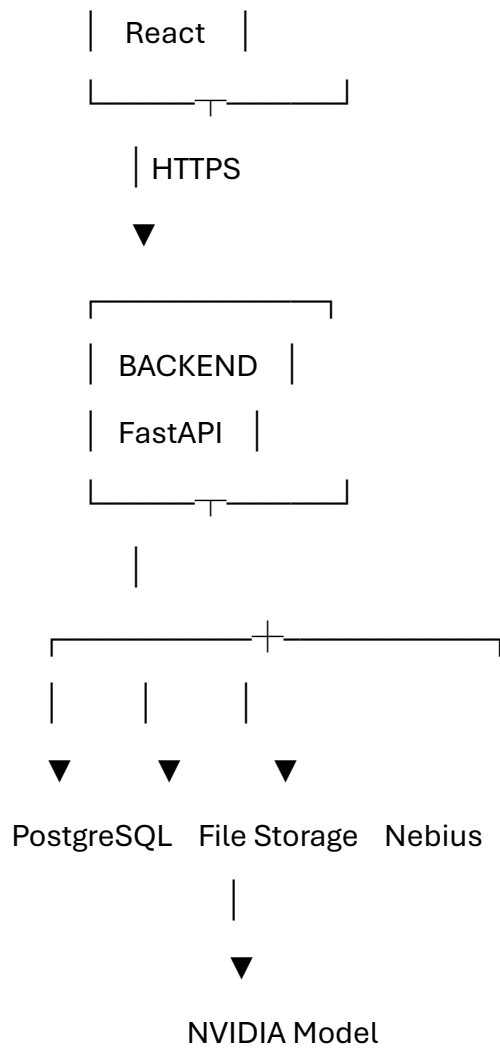
UI

That connected demo is consistent with the concept's proposed three-minute story.

30. Deployment Architecture

Target something conceptually like:





Containerize the backend.

A local development environment can use:

Docker Compose

Frontend

Backend

PostgreSQL

Do not introduce Kubernetes for the hackathon unless the hosting environment genuinely requires it.

31. Development Environments

We should have:

LOCAL

↓

DEVELOPMENT/TEST

↓

PRODUCTION/DEMO

Students develop locally.

Team integrates frequently.

Production is the stable judge-facing environment.

Do not deploy directly to the final demo environment after every random experiment.

32. Git Workflow

Keep it simple.

main

|

└─ stable code

develop

|

├─ feature/chat

├─ feature/database

├─ feature/documents

├─ feature/dashboard

└─ feature/evaluation

Use pull requests.

At least one other team member should review important PRs.

Do not allow everyone to continuously push directly to main.

33. Team Division

Because the team contains different backgrounds, divide work by **workstream**, not by who is "technical enough."

A possible structure:

Product + Scenario + Evaluation

Good for mixed backgrounds.

Responsibilities:

synthetic family

test scenarios

expected outputs

judge test cases

documentation

demo story

evaluation dataset

Frontend

dashboard

chat

timeline

expenses

documents

judge mode

Backend + Database

FastAPI

PostgreSQL

models

services

APIs

repositories

Agent + NVIDIA

Nebius/NVIDIA integration

prompts

structured outputs

tool calling

reference resolution

context reasoning

Integration + Deployment

Shared responsibility led by technically stronger students.

Nobody should work completely alone.

34. Development Sequence

Do **not** start with:

"Let's build an AI agent."

Use this order:

1. Freeze MVP
2. Create synthetic scenarios
3. Create UI wireframes
4. Create database
5. Build backend APIs
6. Build basic frontend
7. Verify frontend ↔ backend ↔ database

8. Connect NVIDIA model

9. Add tools

10. Add household state/context

11. Add dependency reasoning

12. Add priority engine

13. Add PDF/image processing

14. Add background processing

15. Add judge scenarios

16. Automated testing

17. Deploy

18. Break it intentionally

19. Fix it

20. Polish UI

21. Documentation

22. Demo/video

This follows the existing concept's recommendation to establish scenarios, tools/database and core behaviour before polishing the UI.

35. Timeline to October 25

September 18–21

Architecture & planning

Complete:

Blueprint understanding

MVP freeze

UI wireframes

Database schema

GitHub repository

Team responsibilities

September 22–26

Foundation

Complete:

React skeleton

FastAPI skeleton

PostgreSQL

Database models

Basic CRUD APIs

Synthetic family

Core scenarios

September 27–October 4

First end-to-end product

Target:

User → React

→ FastAPI

→ PostgreSQL

→ UI

and basic NVIDIA connectivity.

October 5–11

Agentic core

Complete:

NVIDIA integration

Structured extraction

Tool calling

Context retrieval

Commitment creation/update

Reference resolution

October 12–16

Differentiating features

Complete:

Dependencies

Priority engine

Document upload

Image/PDF processing

Human confirmation

Background processing

October 17–19

Full integration

All major features should work together.

Feature development should effectively stop here.

October 20–22

Testing

Automated tests

Agent evaluation

Failure cases

Duplicate handling

Ambiguity

UI bugs

Deployment testing

October 23–25

Product freeze

README

Architecture diagram

Evaluation results

Demo data

Judge mode

Demo script

Video preparation

Final deployment

Backup demo plan

Do not plan to finish core coding on October 25.

36. Definition of Done

The product is **not done because the chatbot answers questions.**

It is done when all of these work together:

✓ Text input

✓ PDF/image upload

✓ NVIDIA model genuinely used

✓ Structured extraction

✓ Tool calling

✓ PostgreSQL persistence

✓ Expenses

✓ Commitments

✓ State updates

✓ Dependency relationships

✓ Priority reasoning

✓ Clarification when uncertain

✓ Completion tracking

✓ Duplicate prevention

✓ Background deadline checks

✓ Dashboard

✓ Timeline

✓ Judge scenarios

✓ Automated tests

✓ Evaluation results

✓ Error handling

✓ Logging

✓ Public deployment

✓ README

✓ Architecture diagram

✓ Demo video

37. What NOT to Build Before October 25

Do not add:

WhatsApp

Voice

Bank integration

UPI

Automatic payments

Real hospital integration

Real college integration

Investment advice

Medical diagnosis

IoT

Complex mobile application

20 separate agents

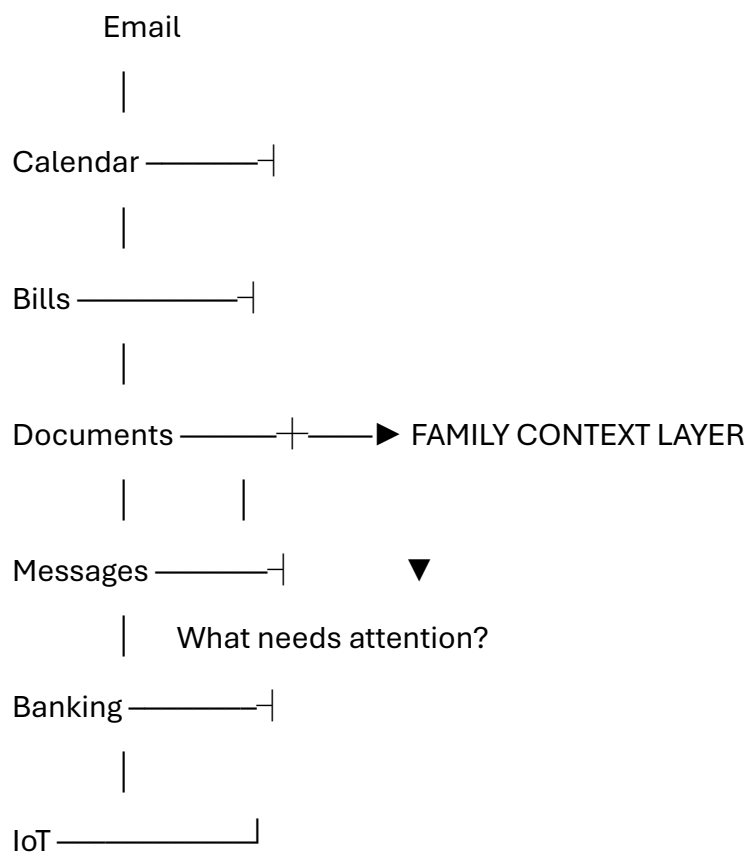
Kubernetes just for appearance

These can become future-work slides.

The concept already explicitly excludes these from the pre-deadline scope.

38. Future Product Architecture

If this becomes a real product later:



Potential future capabilities already identified include shared calendar, document-expiry tracking, subscriptions, maintenance history, elder-care logistics, budgeting, email extraction, permissions, recurring commitments, voice and multilingual input.

But none of these should distract the current team.

39. Four Flows Every Student Must Understand

A student does **not** need to understand every line of backend code.

But every team member should understand these four flows.

Flow A — Text

"₹1,850 groceries today"

↓

Frontend

↓

FastAPI

↓

Family Context Agent

↓

NVIDIA Model

↓

Expense Tool

↓

PostgreSQL

↓

Dashboard

Flow B — Document

Electricity Bill PDF

↓

Frontend

↓

Backend

↓

File Storage

↓

Document Processing

↓

NVIDIA Model

↓

Extracted Information

↓

Human Confirmation

↓

Commitment Tool

↓

PostgreSQL

Flow C — Proactive Attention

Scheduled Job

↓

Active Commitments

↓

Deadline + Dependency Check

↓

Priority Engine

↓

Notification

↓

Dashboard

Flow D — State Update

"Paid electricity bill"

↓

Agent

↓
Find Existing Commitment

↓
Update

↓
COMPLETED

↓
Close Notification

↓
Recalculate Priorities

↓
Dashboard Changes

If a commerce-background student can explain these four flows, they understand the heart of the application.

40. Very Small Student Discussion

Only three questions are necessary at this stage.

Question 1: Look at the proposed dashboard. Is there anything important a normal family would want to see immediately that we have missed?

Question 2: Think of one realistic household situation where two activities depend on each other, similar to CBC → Doctor Review.

Question 3: During testing, try to find one situation where the agent should **ask the user instead of guessing**.

That's enough.

They do **not** need to research the architecture and redesign PostgreSQL, FastAPI, scheduling, storage, agent orchestration or deployment from scratch.

Final Product Principle

The strongest technical idea in this project is not the chatbot.

It is this:

SCATTERED FAMILY INFORMATION

|



UNDERSTAND WITH AI

|



STRUCTURED STATE

|



CONNECT RELATIONSHIPS

|



TRACK CHANGE OVER TIME

|



DETERMINE ATTENTION

|



HELP THE FAMILY

A normal chatbot responds to what the user says **now**.

The Family Context Agent should understand what the family said **before**, what is true **now**, what is expected **later**, how those things are **connected**, and what requires **attention next**.

That is the capability we need to prove to the judges.

And that is why the existing concept's final warning is particularly important: the finished system must demonstrate a **context-coordination agent**, not simply reminders + expense tracking + chat packaged into one application.

This blueprint should now be treated as the team's initial engineering baseline.

Architecture changes are allowed when implementation proves a better approach, but

changes should have a clear reason rather than being made simply because somebody found another framework online.